Session 4: From Hand-Made Filters to CNNs and YOLO

There are 15 notebooks in `Session4/`. They're reordered here into one story:

> **Part A:** hand-designed filters and a classic detector (Haar) → **Part B:** hand-designed *feature vectors* (ORB, HOG) → **Part C:** *learned* filters (CNNs) → **Part D:** a complete deep detector (YOLO)

Part	Folder / notebooks
A	`Convolutional_Filters_Edge_Detection/` 1 → 2 → 3 → 4
B	`Feature_Vectors/` 1 (ORB) → 2_1 (HOG) → 2_2 (HOG examples)
C	`CNN_Layers/` 1 → 2 → 3 → 4_1 → 4_2 → 4_3 → 5_1 → 5_2
D	`YOLO/YOLO.ipynb` with `darknet.py` and `utils.py`

> 🎯 Only part A is close to the project. Parts C and D need PyTorch, which isn't in your `ml_lab` environment. Install it with `pip install torch torchvision` if you want to run them.

Part A: hand-made filters

A1. `1. Finding Edges and Custom Kernels`

Cell 2: loading with matplotlib

```
import matplotlib.image as mpimg
image = mpimg.imread(brain_image)
```

- `mpimg.imread` (matplotlib) returns **RGB**, unlike OpenCV, which returns BGR.
- That's why the next cell correctly uses `cv2.COLOR_RGB2GRAY`.

Cell 6: building kernels by hand

```
sobel_y = np.array([[-1,-2,-1],
                    [ 0, 0, 0],
                    [ 1, 2, 1]])        # bottom minus top → horizontal edges
sobel_x = np.array([[-1, 0, 1],
                    [-2, 0, 2],
                    [-1, 0, 1]])        # right minus left → vertical edges
gaussian_blur = 1/16 * np.array([[1,2,1],
                                 [2,4,2],
                                 [1,2,1]])   # weights sum to 16, /16 → 1
blured_image   = cv2.filter2D(gray, -1, gaussian_blur)
filtered_image = cv2.filter2D(blured_image, -1, sobel_y)
```

- The 3×3 Gaussian here is a small approximation built from binomial numbers (1 2 1).
- The image is blurred first, then the edge filter is applied. Session 2 explained why: derivatives amplify noise.
- ⚠️ The task says "create a Sobel **x** operator", but the code applies `sobel_y`. Swap it in and compare.
- ⚠️ `-1` (uint8 output) drops negative responses, so only dark→bright edges appear.

Cells 8–9: a 5×5 kernel

- The weights `1 2 4 2 1 …` add up to **100**, which is why it's multiplied by `1/100`.
- A larger blur kernel gives smoother, cleaner edges, and fine detail disappears.

A2. `2. Gaussian Blur`

```
gray_blur = cv2.GaussianBlur(gray, (5, 5), 0)          # sigma 0 → computed from ksize
filtered         = cv2.filter2D(gray,      -1, sobel_y)
filtered_blurred = cv2.filter2D(gray_blur, -1, sobel_y)
retval, binary_image = cv2.threshold(filtered_blurred, 20, 255, cv2.THRESH_BINARY)
```

- `np.copy(image)` (cell 2) makes a separate copy so the original isn't changed.

- The comparison shows **edges on the raw image vs on the blurred image**. The blurred version has much less speckle.
- Thresholding at 20 turns the edge strengths into a black-and-white edge map.
- ⚠️ In cell 6 the left panel is titled "original gray", but it actually shows the *filtered* raw image.

A3. `3. Canny Edge Detection`

```
wide  = cv2.Canny(gray, 30, 100)    # low thresholds → many edges, including weak ones
tight = cv2.Canny(gray, 200, 240)   # high thresholds → only the strongest outlines
```

- See Session 2, part 5, for the five stages of Canny.
- **Sunflower exercise (cell 6):** `lower = 290`, `upper = 200`.
  - ⚠️ These are reversed. OpenCV quietly swaps them, so it really runs (200, 290).
  - ⚠️ The comment says "small and low", which no longer matches the values.
  - The task: find values that keep the flower's outline and drop the background texture.

A4. `4. Haar Cascade, Face Detection`

💡 **Viola–Jones (2001)**, the first real-time face detector:

- **Haar features** are black-and-white rectangle patterns, for example "the eye region is darker than the cheeks below it".
- An **integral image** lets the sum of any rectangle be computed in 4 lookups, which makes the features very cheap.
- **AdaBoost** picks the few thousand best features out of a huge pool.
- The **cascade** is a series of stages. Early stages are tiny and quickly reject most non-face windows; only promising windows reach the later, more expensive stages. That's what makes it fast.

```
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
gray  = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
face_cascade =
cv2.CascadeClassifier('detector_architectures/haarcascade_frontalface_default.xml')
faces = face_cascade.detectMultiScale(gray, 4, 6)
for (x, y, w, h) in faces:
    cv2.rectangle(img_with_detections, (x, y), (x+w, y+h), (255, 0, 0), 5)
```

- ⚠️ The image is already RGB but is converted with `BGR2GRAY`. That swaps the red and blue weights; it's a minor effect, but `RGB2GRAY` is correct.
- The `.xml` file holds the trained cascade. OpenCV also ships its own copy at `cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'`.
- `detectMultiScale(image, scaleFactor, minNeighbors)`:
  - `scaleFactor`: how much the image shrinks between search levels. ⚠️ `4` is extremely coarse (typical values are 1.05–1.3), so faces of some sizes can be skipped entirely.
  - `minNeighbors`: how many overlapping raw hits are needed to accept a face. Higher means fewer false positives.
- It returns an array of `(x, y, w, h)` boxes, where `(x, y)` is the top-left corner.
- `cv2.rectangle(img, corner1, corner2, color, thickness)`: `(255,0,0)` on this **RGB** image is red.

Part B: hand-designed feature vectors

B1. `Feature_Vectors/1. ORB`

This goes deeper than Session 3, testing each of ORB's invariances one at a time.

Cell 2: loading

- The face image is loaded and converted to RGB for display and gray for analysis.
- `plt.rcParams['figure.figsize'] = [20,10]` sets the default plot size for every later figure.

Cell 3: `ORB_create` parameters (read this table)

Parameter	Default	Meaning
`nfeatures`	500	Maximum number of keypoints to keep
`scaleFactor`	1.2	Shrink factor between pyramid levels (> 1)
`nlevels`	8	Number of pyramid levels
`edgeThreshold`	31	Border width where nothing is detected (≥ `patchSize`)
`WTA_K`	2	Points compared per descriptor element. 2 → `NORM_HAMMING`; 3 or 4 → `NORM_HAMMING2`
`scoreType`	HARRIS	How keypoints are ranked
`patchSize`	31	Size of the patch BRIEF looks at

Cell 4: detecting and drawing

```
orb = cv2.ORB_create(200, 2)                    # nfeatures=200, scaleFactor=2
keypoints, descriptor = orb.detectAndCompute(training_gray, None)
cv2.drawKeypoints(training_image, keypoints, keyp_without_size, color=(0,255,0))
cv2.drawKeypoints(training_image, keypoints, keyp_with_size,
                  flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
```

- `copy.copy(image)` creates output images to draw on. The 3rd argument of `drawKeypoints` is the image to draw onto.
- **Rich keypoints** show a circle (the patch size) and a radius line (the orientation).

Cell 7: matching an image with itself

```
orb = cv2.ORB_create(1000, 2.0)
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = sorted(bf.match(descriptors_train, descriptors_query), key=lambda x: x.distance)
result = cv2.drawMatches(training_gray, keypoints_train, query_gray, keypoints_query,
                         matches[:300], query_gray, flags=2)
```

- When an image is matched with itself, every keypoint should match. This is a sanity check.
- The 6th argument (`query_gray`) is the output image. `flags=2` is `NOT_DRAW_SINGLE_POINTS`.
- ⚠️ `cvtColor(training_image, COLOR_BGR2GRAY)` is applied to an **RGB** image, so the gray weights are slightly off. The effect is harmless.

Cells 10–17: invariance tests

Test	Query image	What happens	💡 Why
Scale	`faceQS.png` , ¼ size	Only 65 keypoints vs 831, but most of them match	The pyramid searches several sizes
Rotation	`faceR.jpeg` , 90°	About 78% match	Each keypoint gets an orientation, and BRIEF is rotated to it
Brightness	`faceRI.png` , brighter	About 63% match	BRIEF compares *pairs* of pixels; a uniform brightness change doesn't change which one is brighter
Noise	`faceRN5.png`	About 63% match, using `scaleFactor=1.3`	Smoothing inside BRIEF; a finer pyramid helped here

Cells 18–23: finding the face in a group photo

- `ORB_create(5000, 2.0)` : the group photo is big and busy, so many more features are allowed.
- The best 85 matches cluster on the woman's face. That's **detection by matching**.
- To get an actual **box** around her, you'd add `findHomography` with RANSAC from Session 3 and transform the training image's corners.

B2. `Feature_Vectors/2_1. HOG` (Histogram of Oriented Gradients)

💡 **Why HOG exists:** ORB works on objects with consistent internal details (eyes, nose). People and cars differ too much inside, but their **outline shape** is consistent. HOG describes that shape.

The algorithm (cell 1)

1. Pick a detection window, for example 64×128 for a standing person.
2. Compute each pixel's gradient magnitude and direction (Sobel, Session 2).
3. Split the window into **cells**, for example 8×8 pixels.
4. In each cell, build a **histogram of directions**: 9 bins covering 0–180°. Each pixel votes with its **magnitude**.
5. Group cells into **blocks**, for example 2×2 cells, which **overlap** by moving one cell at a time.
6. **Normalize** each block. This is what makes it robust to contrast and lighting.
7. Concatenate everything into one long vector: the **HOG descriptor**.
8. Train a classifier (an **SVM**) on many positive and negative examples.
9. Slide the window over new images and ask the SVM "person or not?".

Cell 4: loading

- The triangle tile image is loaded and converted to RGB and gray. The shapes are printed.

Cell 6: setting the parameters

```
cell_size = (6, 6)
num_cells_per_block = (2, 2)
block_size = (2*6, 2*6)                          # (12, 12) pixels
x_cells = gray_image.shape[1] // cell_size[0]    # number of cells across
y_cells = gray_image.shape[0] // cell_size[1]    # number of cells down
block_stride = (6*1, 6*1)                        # move 1 cell at a time → overlap
num_bins = 9
win_size = (x_cells*6, y_cells*6)                # whole image, cropped to a multiple of 6
hog = cv2.HOGDescriptor(win_size, block_size, block_stride, cell_size, num_bins)
hog_descriptor = hog.compute(gray_image)
```

- `.shape[1]` is the **width** and `.shape[0]` is the **height**. `//` is integer division.
- The window, block and stride must all be **whole multiples of the cell size**.
- `compute` returns one long vector of numbers.

Cell 8: checking the length

```
blocks_x = (x_cells − cells_per_block_x) / stride + 1
total    = blocks_x × blocks_y × 2 × 2 × 9
```

- For example, 10 cells with blocks 2 wide and a stride of 1 gives positions 0..8, so **9 blocks**.
- Each block contributes 2×2 cells × 9 bins = 36 numbers.
- The printed shape should equal `total`.

Cell 10: visualization (interactive)

- **Reshape** the vector into `(blocks_x, blocks_y, 2, 2, 9)`, then `.transpose(1,0,2,3,4)`. OpenCV stores the blocks column by column, and this makes rows come first so the layout matches the image.
- **Average:** each cell appears in up to 4 overlapping blocks. The loops add all of its histograms into `ave_grad` and divide by `hist_counter`.
- **Bins become arrows:** `deg = linspace(0, π, 9)`. Each bin's height becomes the length of an arrow pointing in that bin's direction, using `cos` and `sin` for the U and V components. X and Y place each arrow at its cell's center.
- `quiver` draws all the arrows. Headless white lines on a black background give the classic HOG picture.
- `onpress(event)`: a mouse-click handler. It finds the clicked cell, draws a magenta rectangle on it, zooms in, and shows that cell's histogram as a bar chart.
  `fig.canvas.mpl_connect('button_press_event', onpress)` attaches the handler.
- ⚠️ `%matplotlib notebook` only works in the classic Jupyter Notebook. In VS Code or JupyterLab, install `ipympl` and use `%matplotlib widget` instead, otherwise clicking does nothing.

Cell 11: reading the histograms

- **Inside** a triangle: there's no dominant direction, just small random bars.
- **Horizontal edge:** the brightness changes vertically, so the **90°** bin dominates.
- **Vertical edge:** the brightness changes horizontally, so the bars are near 0° and 180°. Those are the **same direction**, because HOG uses *unsigned* gradients (0–180°), so the votes split between the **10°**

and **170°** bins.

- **Diagonal edge:** a 40° gradient lies between the 30° and 50° bin centers, so its vote is **shared** between them. That's why two bins stand out.

B3. `2_2. HOG Examples`

- It's the same code as 2_1 in one cell, with the settings at the top: the image, `cell_size`, `num_cells_per_block` and `num_bins`.
- For example, `cell_size=(50,50)` on `jeep2.jpeg` shows only coarse shape, while small cells show fine detail but also more noise.
- **Try it:** use the same image with cell sizes 8, 16 and 32, and with 9 vs 18 bins.

Part C: CNNs, filters that *learn*

💡 **The big idea:** in Part A, people designed the kernels (Sobel and so on). A **CNN** (convolutional neural network) has the same convolution layers, but **the kernel numbers are learned from data**.

C1. `1. Conv Layer Visualization`

Cell 2: preparing the image

```
gray_img = cv2.cvtColor(bgr_img, cv2.COLOR_BGR2GRAY)
gray_img = gray_img.astype("float32") / 255
```

- Neural networks work with floats in a small range, so 0–255 is scaled to **0–1**.

Cells 4–6: four hand-made filters

```
filter_vals = np.array([[-1,-2, 2, 1]] * 4)   # dark on the left, bright on the right
filter_1 = filter_vals        # responds to left→right dark-to-bright (vertical edges)
filter_2 = -filter_1          # the opposite polarity
filter_3 = filter_1.T         # transposed → top-to-bottom (horizontal edges)
filter_4 = -filter_3
filters = np.array([filter_1, filter_2, filter_3, filter_4])   # shape (4, 4, 4)
```

- The plotting loop writes each weight's number on its square. The text is white on negative squares and black on positive ones.

Cell 8: a one-layer network in PyTorch

```
class Net(nn.Module):
    def __init__(self, weight):
        super(Net, self).__init__()
        k_height, k_width = weight.shape[2:]
        self.conv = nn.Conv2d(1, 4, kernel_size=(k_height, k_width), bias=False)
        self.conv.weight = torch.nn.Parameter(weight)
    def forward(self, x):
        conv_x = self.conv(x)
        activated_x = F.relu(conv_x)
        return conv_x, activated_x
weight = torch.from_numpy(filters).unsqueeze(1).type(torch.FloatTensor)
```

- `nn.Module` is the base class for every PyTorch model. Layers go in `__init__`, and the calculation goes in `forward`.
- `super().__init__()` sets up the parent class. It's required.
- `nn.Conv2d(in_channels=1, out_channels=4, kernel_size=4, bias=False)`: 1 gray input channel, 4 filters, and therefore 4 output images.
- Conv2d weights must have shape **(out, in, kH, kW)** = (4, 1, 4, 4). `unsqueeze(1)` inserts the missing "in" dimension.

- Normally the weights would be random and learned. Here they're **set by hand** to our filters.
- **ReLU** means `max(0, x)` : negative values become 0.
- 💡 PyTorch's `Conv2d` and OpenCV's `filter2D` compute the same thing (correlation).

Cells 10–13: running and viewing the result

```
gray_img_tensor = torch.from_numpy(gray_img).unsqueeze(0).unsqueeze(1)   # (1, 1, H, W)
conv_layer, activated_layer = model(gray_img_tensor)
viz_layer(conv_layer)          # before ReLU
viz_layer(activated_layer)     # after ReLU
```

- PyTorch images are **NCHW**: batch, channels, height, width. Two `unsqueeze` calls turn (H, W) into (1, 1, H, W).
- `viz_layer` plots `layer[0, i]` , which is image 0 of the batch, channel i. `.data.numpy()` converts it back to NumPy.
- With no padding, the output is `H - 4 + 1` pixels tall.
- 💡 **After ReLU** only one polarity of edge remains, much like the `uint8` clipping in OpenCV. That's why the filters come in ± pairs.

C2. `2. Pool Visualization`

```
self.pool = nn.MaxPool2d(4, 4)    # kernel 4×4, stride 4
pooled_x = self.pool(activated_x)
```

- **Max pooling** takes each 4×4 block and keeps only its **largest** value, so the image becomes 4× smaller in each direction.
- 💡 Why pool?
  - Less data means faster later layers.
  - The strongest responses are kept.
  - It adds a little **translation tolerance**: a feature that moves by a pixel still gives the same max.

C3. `3. Load and Visualize FashionMNIST`

```
data_transform = transforms.ToTensor()
train_data = FashionMNIST(root='./data', train=True, download=True, transform=data_transform)
train_loader = DataLoader(train_data, batch_size=20, shuffle=True)
images, labels = next(iter(train_loader))
```

- **FashionMNIST** has 60,000 training and 10,000 test images. Each is 28×28 gray and belongs to one of 10 clothing classes.
- `download=True` downloads the data into `./data` the first time, so it needs internet once.
- `ToTensor()` converts a PIL image (H, W, uint8 0–255) into a tensor (1, H, W, float 0–1).
- **DataLoader** delivers the data in **batches** of 20, shuffled. `iter` and `next` grab one batch:
  - `images` has shape (20, 1, 28, 28).
  - `labels` has shape (20,), containing class numbers 0–9.
- `classes[labels[idx]]` turns a number into a name.
- Cell 8 zooms into one image and writes each pixel's value on it.

- ⚠️ The "normalization" text is broadly right, but `ToTensor` only **scales** values to 0–1. Real normalization would add `transforms.Normalize(mean, std)`.

C4. `4_1. Classify FashionMNIST, exercise` (your own network)

Cell 7 (broken) and cell 8 (fixed)

```
self.layer1 = nn.Sequential(OrderedDict([
    ('conv1', nn.Conv2d(1, 10, kernel_size=3)),
    ('relu1', nn.ReLU()),
    ('bachnorm1', nn.BatchNorm2d(10))]))
```

- `nn.Sequential` runs layers in order. `OrderedDict` gives each one a **name**, so later you can write `net.layer1.conv1`.
- ⚠️ **Cell 7 crashes:** `conv1` outputs **5** channels, but `BatchNorm2d(10)` and `conv2` expect **10**. Cell 8 redefines the class with 10, which fixes it.
- **Shapes as the data flows through cell 8:**

Layer	Output shape
input	1 × 28 × 28
conv1 (3×3) → ReLU → BatchNorm	10 × 26 × 26
maxpool 2	10 × 13 × 13
conv2 (3×3) → ReLU	20 × 11 × 11
flatten (`out.view(out.size(0), -1)`)	2420
fc1 → ReLU → Dropout 0.4	200
fc2 → ReLU → Dropout 0.4	100
fc3 → **ReLU**	10
`log_softmax`	10 log-probabilities

- **Conv output size** = `(W - F)/S + 1`, for example (28 − 3)/1 + 1 = 26.
- `view(batch, -1)` flattens each image into one vector. `-1` means "work out this size".
- **BatchNorm** rescales each channel to a stable range across the batch, which makes training faster and steadier.
- **Dropout(0.4)** randomly switches off 40% of the values during training, so the network can't rely on any single neuron. This reduces **overfitting**.
- ⚠️ A **ReLU on the final layer** (`relu5`) forces every class score to be ≥ 0. Classes whose score gets stuck at 0 can't learn properly. Remove the ReLU from the last layer.

Cells 10–12: loss, optimizer, and accuracy before training

```
criterion = nn.NLLLoss()                    # pairs with log_softmax output
optimizer = optim.Adam(net.parameters())    # default lr = 0.001
_, predicted = torch.max(outputs.data, 1)  # index of the highest score = the predicted class
correct += (predicted == labels).sum()
```

- `NLLLoss` together with `log_softmax` is mathematically identical to `CrossEntropyLoss` applied to raw scores.
- `torch.max(x, 1)` returns (the max values, their indices) along dimension 1, the classes. Only the indices are needed.
- Before training, accuracy is about 10%, the same as random guessing among 10 classes.
- 💡 Evaluation should be wrapped in `with torch.no_grad():` so it doesn't waste memory recording gradients.

Cell 14: the training loop (memorize these 5 lines)

```
optimizer.zero_grad()              # 1. clear old gradients (they accumulate otherwise)
outputs = net(inputs)              # 2. forward pass
loss = criterion(outputs, labels)  # 3. how wrong was it?
loss.backward()                    # 4. compute gradients (backpropagation)
optimizer.step()                   # 5. nudge the weights
```

- `loss.item()` converts a 1-element tensor to a plain Python number.
- `batch_i % 1000 == 999` prints the average loss every 1000 batches.
- An **epoch** is one full pass over all 60,000 images. With a batch size of 10, that's 6,000 batches per epoch, × 20 epochs, which is **slow on a CPU**.

Cell 17: testing

```
net.eval()
test_loss = test_loss + ((torch.ones(1) / (batch_i + 1)) * (loss.data - test_loss))
correct = np.squeeze(predicted.eq(labels.data.view_as(predicted)))
for i in range(batch_size):
    class_correct[label] += correct[i].item(); class_total[label] += 1
```

- `net.eval()` **switches off dropout** and makes BatchNorm use its stored averages. ⚠️ Call `net.train()` again before any further training.
- The `test_loss` line is a **running average**: new average = old + (new value − old) / count.
- `.eq` compares element by element and gives True or False for each image.
- Per-class accuracy shows *which* clothes get confused, such as shirts vs T-shirts vs coats.
- ⚠️ `range(batch_size)` assumes every batch is full. That works because 10,000 is divisible by 10.

Cells 19 and 23–25: visualize, save, load

- Prediction titles are **green** when correct and **red** when wrong, shown as `pred (true)`.
- `torch.save(net.state_dict(), path)` saves **only the weights**. ⚠️ The `saved_models/` folder must already exist.
- To load, create the **same** `Net()` class first, then call `net.load_state_dict(torch.load(path))`.

C5. `4_2. Solution 1` vs `4_3. Solution 2`

Both use 2 conv layers (10 then 20 filters, each followed by a pool), so the data is 20×5×5 = **500** values after flattening. (The second pool turns 11 into 5.5, which rounds down to 5.)

	Solution 1	Solution 2
FC layers	500 → **10** (with ReLU ⚠️), then `log_softmax`	500 → 50 → Dropout(0.4) → **10 raw scores**
Loss	`NLLLoss`	`CrossEntropyLoss` (does the softmax itself) ✅
Optimizer	SGD, lr 0.001	SGD, lr 0.001, **momentum 0.9**
Weakest class	Shirt and pullover around 0%	Shirt around 60%

- 💡 **Momentum** keeps part of the previous step's direction, like a rolling ball. It speeds up training and helps avoid getting stuck.
- 💡 Solution 2 is better because of **dropout**, **momentum**, and **no ReLU on the output**.
- Cell 16 plots the loss over time. It should fall quickly at first, then flatten.

C6. `5_1. Feature viz` and `5_2. Visualize Your Net`

```
net.load_state_dict(torch.load('saved_models/fashion_net_ex.pt'))
w = net.conv1.weight.data.numpy()               # shape (10, 1, 3, 3)
plt.imshow(w[i][0], cmap='gray')                # the i-th learned 3×3 kernel
c = cv2.filter2D(img, -1, w[i][0])              # apply it to a test image
```

- These notebooks show what the network **learned**. Many first-layer kernels turn out to look like Sobel edge detectors, which nobody designed.
- `filter2D` on a **float32** image keeps negative values, so these activation maps are correct for conv1.
- The plotting loop alternates: even subplot positions show a kernel and odd ones show its result (`i % 2`).
- ⚠️ **conv2 weights** have shape (20, **10**, 3, 3). Each conv2 filter has a separate slice for each of conv1's 10 output maps. The code shows only slice `[0]` and applies it to the *raw image*, but the real conv2 sees conv1's maps after ReLU and pooling and **adds up all 10 slices**. Treat those conv2 pictures as illustrations only.
- `5_2` is the same notebook using **your** exercise network (`net.layer1.conv1` , named through `OrderedDict`) and `model_91.pt` .

Part D: YOLO (You Only Look Once)

💡 **The idea:** older detectors classified thousands of windows one by one. YOLO looks at the image **once**:

- It divides the image into a **grid**. Each grid cell predicts a few boxes, using **anchors** (preset box shapes), plus class probabilities.
- **YOLOv3** predicts at **3 scales**: 13×13, 26×26 and 52×52 grids for a 416×416 input. That covers large, medium and small objects.
- It has 3 anchors per cell and **80 COCO classes**, with the Darknet-53 backbone.
- Each prediction contains x, y, w, h, an **objectness** score, and class probabilities.

Cells 2–6: setup

```
from utils import *
from darknet import Darknet
m = Darknet('./cfg/yolov3.cfg')            # build the architecture from a text description
m.load_weights('./weights/yolov3.weights') # ~240 MB of pre-trained COCO weights
class_names = load_class_names('data/coco.names')
m.print_network()
```

- `from utils import *` imports every function from `utils.py`. That's convenient but hides where names come from.
- The `.cfg` file lists the layers (conv, shortcut, upsample, yolo), and `Darknet` turns it into PyTorch layers.
- ⚠️ `yolov3.weights` must be present in `weights/`. It's usually not in git because it's so large.
- ⚠️ `darknet.py` was written for PyTorch 0.4, so it may need small fixes on modern PyTorch.

Cell 8: resizing

```
resized_image = cv2.resize(original_image, (m.width, m.height))   # (416, 416)
```

- The network needs exactly 416×416. Plain resizing **squashes** the aspect ratio; modern YOLO versions pad (letterbox) instead.

Cells 9–12: the two thresholds ⚠️ the naming is confusing

Variable	What it *really* does
`nms_thresh = 0.6`	**Confidence threshold.** Boxes with confidence < 0.6 are dropped (passed into `model(img, nms_thresh)`).
`iou_thresh = 0.4`	**NMS overlap threshold.** Among the remaining boxes, any box that overlaps a higher-scoring one by IoU > 0.4 is removed.

- **IoU** (intersection over union) = overlap area ÷ combined area. It's 0 for no overlap and 1 for identical boxes. See `boxes_iou` in `utils.py`.
- **NMS** (non-maximum suppression), in `utils.nms`:

1. Sort the boxes by confidence.
2. Keep the best box, and zero out every box overlapping it by more than the threshold.
3. Move on to the next surviving box and repeat.

The result is one box per object. ⚠️ This version ignores class, so a "person" box can delete an overlapping "dog" box.

Cell 14: detecting objects

```
boxes = detect_objects(m, resized_image, iou_thresh, nms_thresh)
print_objects(boxes, class_names)
plot_boxes(original_image, boxes, class_names, plot_labels=True)
```

Inside `detect_objects`:

- `img.transpose(2,0,1)` turns HWC into CHW. `.float().div(255.0)` scales to 0–1, and `.unsqueeze(0)` adds the batch dimension, giving (1, 3, 416, 416).
- `model(img, conf)` returns a list of boxes for each of the 3 scales. These are combined and passed to `nms`.
- It takes about 2 seconds per image on a CPU.
- **Box format:** `[cx, cy, w, h, confidence, class_prob, class_id]`. The coordinates are **fractions 0–1** of the image size, which is why the boxes can be drawn on `original_image` at any size.

Cells 17–22: webcam version

- `plot_boxes_image` converts center format to corners: `x1 = (cx - w/2) * width` and so on. It draws a box, then a **filled** rectangle (`thickness=-1`) for the label background, then the text with `putText`.
- `colors` gives each class a random color.
- `showarray` encodes a frame as JPEG and displays it inside the notebook. ⚠️ It's **commented out**, so by default you only see printed text.
- `get_frame` reads a frame and flips it horizontally (`cv2.flip(frame, 1)`) so it acts like a mirror.
- The loop runs until you press **Interrupt**. The `KeyboardInterrupt` is caught, and the camera is released.
- ⚠️ There's no `isOpened()` check. If the camera fails, `frame` is `None` and `cv2.flip` crashes.
- ⚠️ `cls_conf` and `cls_id` are only set when `len(box) >= 7`. Otherwise the next line fails.

🎯 **Why not YOLO for the flag:** it can't run on an ESP32. On a laptop CPU it takes about 2 s per frame, which is far too slow for steering. A colored flag is solved more simply and much faster by HSV thresholding (Sessions 2 and 5).

Session 4 gotchas

1. `mpimg.imread` returns RGB, while `cv2.imread` returns BGR.
2. The Sobel task asks for x but the code uses y; uint8 output clips negative values.
3. The sunflower Canny thresholds are reversed.
4. Haar `scaleFactor=4` is far too coarse; use about 1.1.
5. `%matplotlib notebook` doesn't work in VS Code; use `%matplotlib widget`.
6. Exercise cell 7 has mismatched channel counts (5 vs 10).

7. Don't put ReLU on the final layer.
8. `net.eval()` must be undone with `net.train()` before training again.
9. The conv2 visualization uses only 1 of 10 input slices.
10. In YOLO, `nms_thresh` is really a confidence threshold, and its NMS ignores class.